

music code generation theory

This overall function, `N$Z2`, is responsible for gathering user data and compiling it into a code

```
get N$Z2() {
  if (!this.#Ue) return null
  let e = [
    this.#Ue,
    this.#Ee
  ]
  let s = this.N$52
  for (let t = s.length - 1; t >= 0; t--) {
    let i = s[t]
    let n = i.N$T6?.N$17
    if (!n) continue
    e.push(n, i.N$b2, i.N$A6, i.N$M6, i.N$P6, 0, i.elapsedTime)
  }
  let i
  let n = 0
  t.N$3st.N$a9(function (t) { t: undefined
    if (n < e.length) return e[n++]
    i = t
  })
  return i
}
```

It's not obvious what this is yet tho

`n` is the index, it goes up by 1 per iteration, and is used to extract the value at that position in the array

`e` is the data array, looks like this:

```
>> e
< Array(9) [ "a", 0, "drwyh99xbyyyyyyb", 5, 2, 6, 0, 0, 459 ]
  0: "a"
  1: 0
  2: "drwyh99xbyyyyyyb"
  3: 5
  4: 2
  5: 6
  6: 0
  7: 0
  8: 459
  length: 9
  <prototype>: Array []
```

You can see earlier in the code that `e` starts with

`this.#Ue`, which is the **name**, and

`this.#Ee`, which is likely some versioning which should be kept at `0`

- You can see that at the first line of the function, if the name is not defined, it refuses to generate a code.

Eventually, a for loop occurs, and the rest of the numbers are pushed to it.

In order, these are

- `n`: the settings string, which is basically all your settings but condensed into some string. This is important.
- `i.N$b2`: Type ID (e.g. 5 for chord practice)
- `i.N$A6`: Correct count
- `i.N$M6`: Total count
- `i.N$P6`: Skipped
- `0`: Stays constant
- `i.elapsedTime`: Self explanatory; time in seconds.

More about the loop:

It iterates through `s`, which is `this.N$52`

`s` contains the exercises

What do i mean?

A code can contain info for multiple exercises

If, during an exercise, you change settings and continue, the code takes note of this.

This loop iterates BACKWARDS.

Usually, 7 items are pushed to the array `e`

If you did multiple "exercises" (with different settings without resetting score), you will have more info pushed to `e`, containing information about each segment.

This eventually makes the code longer if you did multiple.

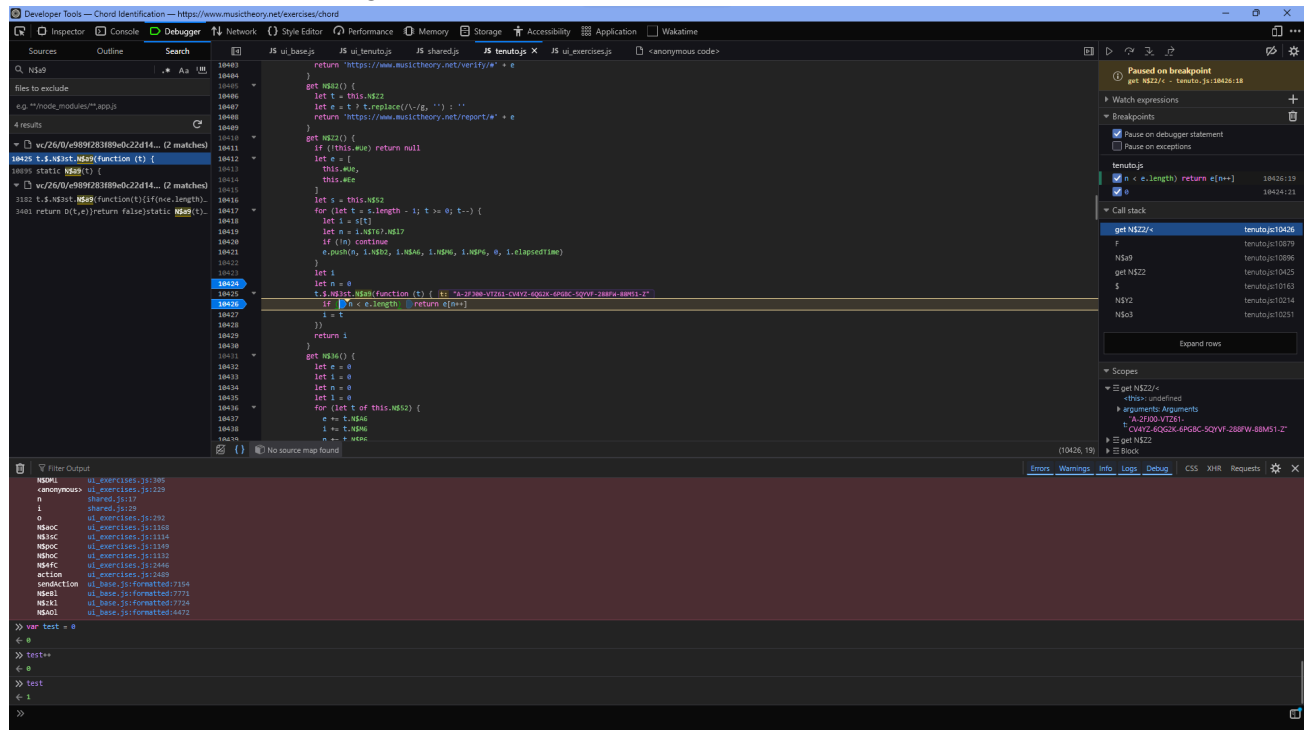
`t.$.N$3st.N$a9` is the function that actually generates the code. here's how it works:

You pass a function to it. This function WILL iterate.

Here's the logic behind it:

- If the current index (`n`) is less than the length of our data array (`e`), then return the value at the current index of `e`, AND add 1 to `n`
 - It's worth noting that running `n++` actually returns the current value of `n`, but upon checking the value of `n` again, it'll be +1 from what it was before.
- Each iteration `i`, which will be returned as the code is assigned as `t`, which is the argument of the iterating function, meaning `N$a9` sets `t` to the code once it has enough information

- This is observed as during the last iteration, t becomes the code



How to leverage this to generate a code:

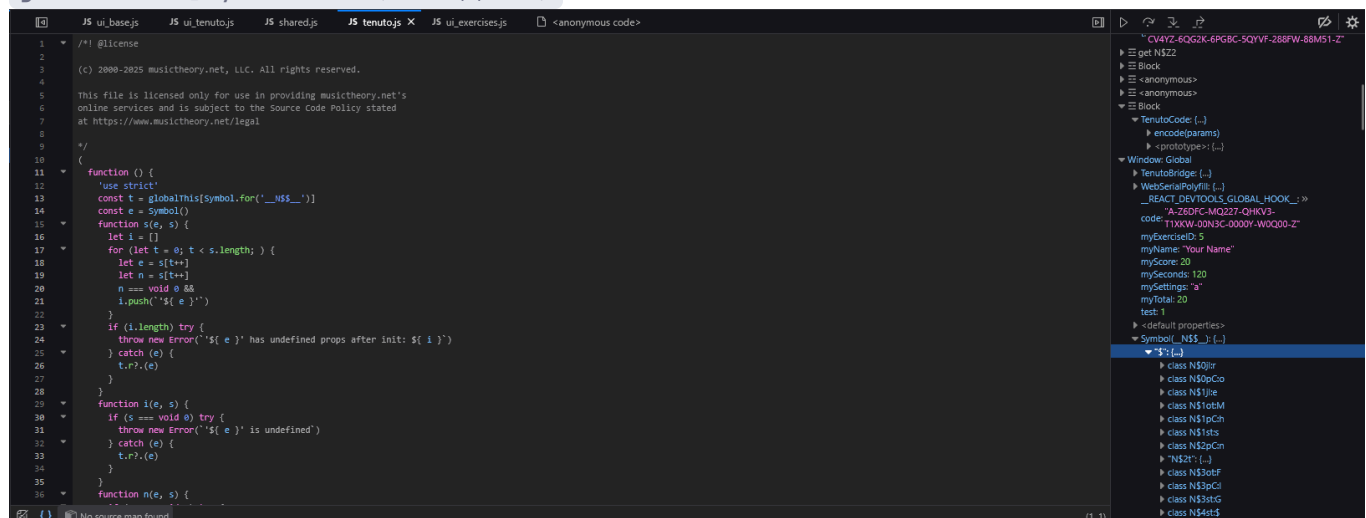
We only need the second part, so we need a way to get to it

If we can pass a "mock" function to `t.$.N$3st.N$a9`, then we win so where is it?

Thankfully, this isn't too hard.

Look at `t`. At the beginning of the file, it's defined as a constant pointing to

`globalThis[Symbol.for("__N$$__")]`



Ok, so now we have a working exploit

All we need to do is pass a function with false information over to the code generating function like so:

Notes: i used an arrow function instead to condense it as much as possible

```

let index = 0;
let payload = [
  "name", // Name
  0, // Version (constant)
  "drwyh99xbYYYYYYb", // Settings string
  5, // Type (5 for chords)
  20, // Correct
  20, // Total
  0, // Skipped
  0, // Constant
  120 // Time in seconds
];
globalThis[Symbol.for("__N$$__")].$.N$3st.N$a9((result) => {
  console.log(result ? result : payload[index]);
  if (index < payload.length) return payload[index++];
});

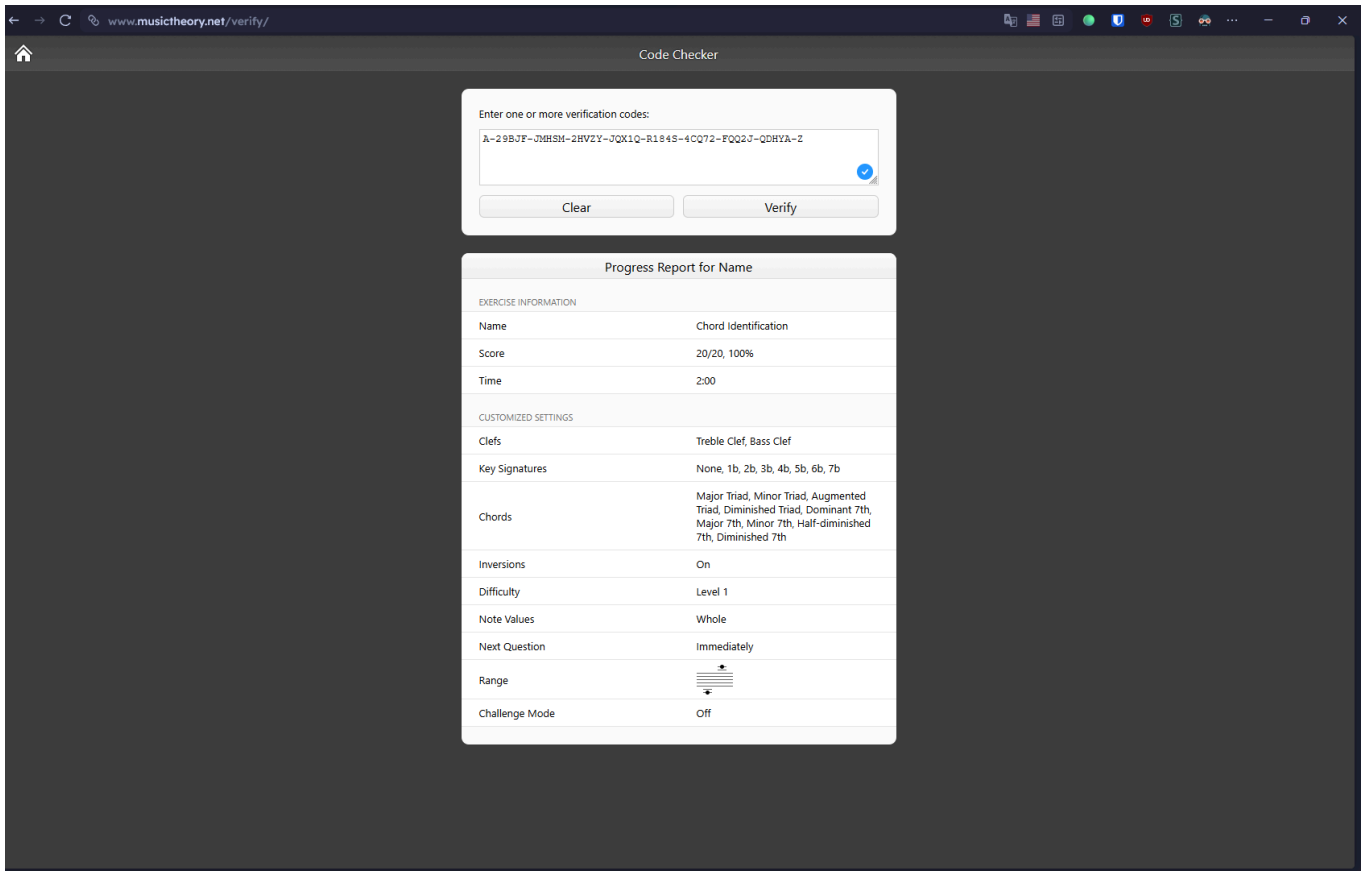
```

Sure enough, we get `A-29BJF-JMHSM-2HVZY-JQX1Q-R184S-4CQ72-FQQ2J-QDHYA-Z`

The screenshot shows a web browser window with the URL `https://www.musictheory.net/exercises/chord`. The developer tools are open, and the console tab is active. The console shows the execution of a JavaScript script that defines a payload array and iterates over it, logging each element. The final output is the chord identification `A-29BJF-JMHSM-2HVZY-JQX1Q-R184S-4CQ72-FQQ2J-QDHYA-Z`.

Log Entry	Source
<code>name</code>	<code>debugger eval code:14:10</code>
<code>0</code>	<code>debugger eval code:14:10</code>
<code>drwyh99xbYYYYYYb</code>	<code>debugger eval code:14:10</code>
<code>5</code>	<code>debugger eval code:14:10</code>
<code>20</code>	<code>debugger eval code:14:10</code>
<code>20</code>	<code>debugger eval code:14:10</code>
<code>0</code>	<code>debugger eval code:14:10</code>
<code>0</code>	<code>debugger eval code:14:10</code>
<code>120</code>	<code>debugger eval code:14:10</code>
<code>undefined</code>	<code>debugger eval code:14:10</code>
<code>A-29BJF-JMHSM-2HVZY-JQX1Q-R184S-4CQ72-FQQ2J-QDHYA-Z</code>	<code>debugger eval code:14:10</code>
<code>← undefined</code>	

AND IT WORKS!!!



Note about the setting string: I don't actually have a way to get this yet, but what you should do is setup a breakpoint in devtools at that function `N$3st`, read the array `e`, and copy it so that you can just use musictheory's ui to get it